Experiment No.: 4

Theory of Thread:

Pthreads, or POSIX threads, serves as a standardized approach to thread creation and synchronization, gaining support across a spectrum of operating systems like Linux, macOS, and iOS. In the realm of C programming, Pthreads offer a pathway for crafting multiple threads within a singular process. This concurrent execution enhances the program's overall efficiency.

In the realm of single-threaded programs, a solitary flow of control prevails, executing tasks sequentially. This sequential execution becomes a bottleneck when a task consumes a considerable amount of time, forcing the program to patiently wait before moving on to the subsequent task. Conversely, the adoption of multi-threaded programs introduces a paradigm shift by enabling multiple tasks to unfold simultaneously in distinct threads. This approach fosters faster program completion times, as the CPU adeptly switches between diverse threads to maintain optimal utilization.

Within the C programming landscape, the Pthreads library becomes instrumental by providing an arsenal of functions dedicated to the creation and management of threads. For instance, the pthread_create function facilitates the birth of a new thread, while the pthread_join function allows for the orderly waiting of a thread's completion. Moreover, Pthreads offers synchronization mechanisms like mutexes and condition variables, acting as guardians to ensure secure access to shared data structures amidst the tumult of multiple threads.

However, treading into the domain of Pthreads in C necessitates vigilance regarding potential race conditions. These conditions arise when multiple threads concurrently access and modify shared data structures, often leading to unexpected outcomes. To avert such pitfalls, judicious use of proper synchronization mechanisms becomes imperative, ensuring a harmonious and error-free execution of multithreaded programs.

The pthreads library in C provides several important functions for creating, managing, and synchronizing threads. Some of the most commonly used functions are:

- **pthread_create**: This function is used to create a new thread. It takes as arguments the address of a pthread_t variable to store the thread identifier, a pointer to the thread's attributes (which can be set using pthread_attr_init), the address of the function that will run as the new thread, and an argument to pass to the thread function.
- **pthread_join**: This function is used to wait for a thread to complete. It takes as arguments the thread identifier and a pointer to a location where the exit status of the thread can be stored.

- **pthread_exit**: This function is used to terminate the calling thread. It takes as an argument a pointer to the exit status of the thread.
- **pthread_mutex_init:** This function is used to initialize a mutex, which is a synchronization primitive that can be used to protect access to shared data structures.
- **pthread_mutex_lock**: This function is used to lock a mutex, preventing other threads from accessing the protected data structure until the mutex is unlocked.
- **pthread_mutex_unlock:** This function is used to unlock a mutex, allowing other threads to access the protected data structure.
- **pthread_cond_init**: This function is used to initialize a condition variable, which is a synchronization primitive that can be used to signal between threads.
- **pthread_cond_wait**: This function is used to wait on a condition variable. It takes as arguments a pointer to a mutex that is associated with the condition variable, and a pointer to the condition variable.
- **pthread_cond_signal:** This function is used to signal a condition variable, waking up a thread that is waiting on it.

Uses of Multithreading:

- User Interface Responsiveness
- Web Servers
- Data Processing and Analysis
- Game Development
- Database Operations
- Network Applications
- Image and Video Processing
- Parallelizing Batch Processing

Problem Statement 1 (Aim):

Write a multithreaded program that calculates various statistical values for a list of numbers. This program will be passed a series of numbers on the command line and will then create three separate worker threads. One thread will determine the average of the numbers, the second will determine the maximum value, and the third will determine the median value.

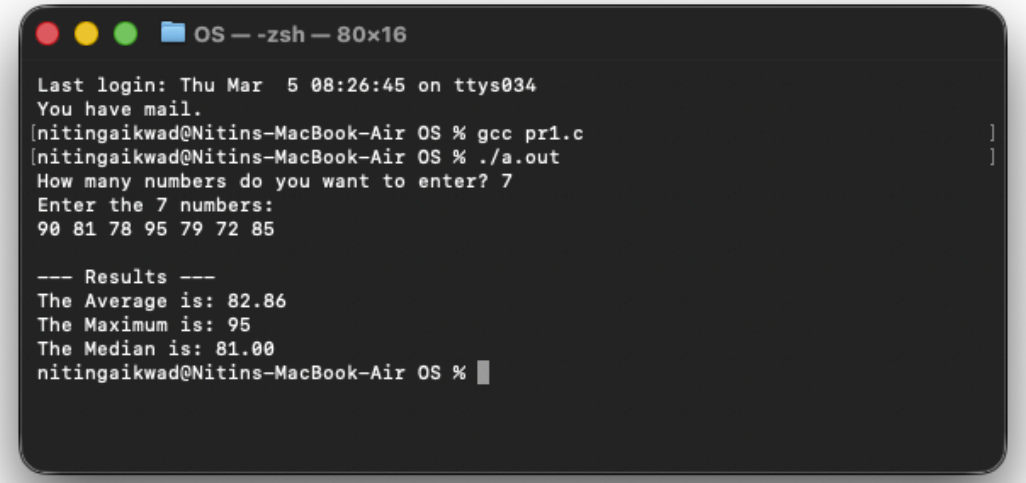
Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

// Global variables shared by all threads
int *numbers;
int count;
double average;
int maximum;
double median;

//  Find the Average
void *calculate_average(void *arg) {
   int sum = 0;
   for (int i = 0; i < count; i++) {
       sum = sum + numbers[i];
   }
   average = (double)sum / count;
```

```c
    pthread_exit(NULL);
}

// Find the Maximum
void *calculate_maximum(void *arg) {
    maximum = numbers[0];
    for (int i = 1; i < count; i++) {
        if (numbers[i] > maximum) {
            maximum = numbers[i];
        }
    }
    pthread_exit(NULL);
}

// Find the Median
void *calculate_median(void *arg) {
    // Copy the array to avoid changing the original order
    int temp[count];
    for(int i = 0; i < count; i++) {
        temp[i] = numbers[i];
    }

    // Sort the array using simple Bubble Sort
    for (int i = 0; i < count - 1; i++) {
        for (int j = 0; j < count - i - 1; j++) {
            if (temp[j] > temp[j+1]) {
                int t = temp[j];
                temp[j] = temp[j+1];
                temp[j+1] = t;
            }
        }
    }

    // Calculate median
    if (count % 2 == 0) {
        median = (temp[count/2 - 1] + temp[count/2]) / 2.0;
    } else {
        median = temp[count/2];
    }
    pthread_exit(NULL);
}

int main() {
    // Ask the user for the total number of elements
    printf("How many numbers do you want to enter? ");
    scanf("%d", &count);

    if (count <= 0) {
        printf("Please enter a valid number.\n");
        return -1;
    }
```

```c
    // Allocate memory for the numbers array
    numbers = (int *)malloc(count * sizeof(int));

    // Take the numbers as input from the user
    printf("Enter the %d numbers:\n", count);
    for (int i = 0; i < count; i++) {
        scanf("%d", &numbers[i]);
    }

    pthread_t t1, t2, t3;

    // Create three separate worker threads
    pthread_create(&t1, NULL, calculate_average, NULL);
    pthread_create(&t2, NULL, calculate_maximum, NULL);
    pthread_create(&t3, NULL, calculate_median, NULL);

    // Wait for all threads to finish their work
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);

    // Print the final calculated values
    printf("\n--- Results ---\n");
    printf("The Average is: %.2f\n", average);
    printf("The Maximum is: %d\n", maximum);
    printf("The Median is: %.2f\n", median);

    // Free the memory
    free(numbers);
    return 0;
}
```

Output



Problem Statement 2

Write a program that creates a single thread that generates random numbers between 1 and 100. The thread should run for a specified number of iterations, printing each generated number to the screen. After the specified number of iterations, the thread.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

void *generate_random(void *arg) {
    int iterations = *((int *)arg);

    for (int i = 0; i < iterations; i++) {
        int random_num = (rand() % 100) + 1;
        printf("Iteration %d: Generated Random Number = %d\n",
i + 1, random_num);
    }
    pthread_exit(NULL);
}

int main() {
    pthread_t thread;
    int iterations = 5;
    srand(time(NULL));

    printf("Starting random number generator thread...\n");
    pthread_create(&thread, NULL, generate_random,
&iterations);
    pthread_join(thread, NULL);
    printf("Thread finished successfully.\n");
    return 0;
}
```

Output:

```
OS — -zsh — 80×16

[nitingaikwad@Nitins-MacBook-Air OS % gcc pr2.c
[nitingaikwad@Nitins-MacBook-Air OS % ./a.out
 Starting random number generator thread...
 Iteration 1: Generated Random Number = 97
 Iteration 2: Generated Random Number = 12
 Iteration 3: Generated Random Number = 66
 Iteration 4: Generated Random Number = 25
 Iteration 5: Generated Random Number = 78
 Thread finished successfully.
 nitingaikwad@Nitins-MacBook-Air OS %
```

Problem Statement 3

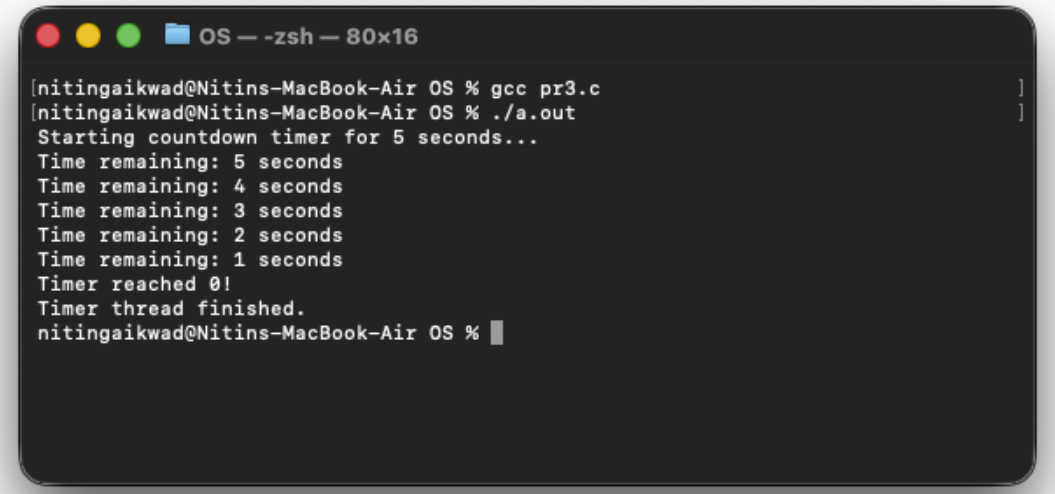
Write a program that creates a single thread that implements a countdown timer. The timer should count down from a specified number of seconds and print the remaining time to the screen every second until the timer reaches 0.

Code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

void *countdown_timer(void *arg) {
    int seconds = *((int *)arg);

    while (seconds > 0) {
        printf("Time remaining: %d seconds\n", seconds);
        sleep(1);
        seconds--;
    }

    printf("Timer reached 0!\n");
    pthread_exit(NULL);
}

int main() {
```

```
    pthread_t thread;
    int seconds = 5;
    printf("Starting countdown timer for %d seconds...\n",
seconds);
    pthread_create(&thread, NULL, countdown_timer, &seconds);
    pthread_join(thread, NULL);

    printf("Timer thread finished.\n");
    return 0;
}
```

Output:



Problem Statement 4

Write a program that creates two threads, each responsible for searching a large text file for a specified keyword. The first thread should search the first half of the file, while the second thread should search the second half. Both threads should report the number of times the keyword was found in their respective sections of the file. The main program should then print the total number of occurrences of the keyword in the entire file.

Code:

```
#include <stdio.h>

#include <pthread.h>

#include <string.h>


// Global variables
```

```c
char words[1000][50];
int total = 0;
char key[] = "system"; // Word to find

int count1 = 0;
int count2 = 0;

// Search the first half
void *search1(void *arg) {
    int mid = total / 2;
    for (int i = 0; i < mid; i++) {
        if (strcmp(words[i], key) == 0) {
            count1++;
        }
    }
    pthread_exit(NULL);
}

// Search the second half
void *search2(void *arg) {
    int mid = total / 2;
    for (int i = mid; i < total; i++) {
        if (strcmp(words[i], key) == 0) {
            count2++;
        }
    }
    pthread_exit(NULL);
}

int main() {
    // Open and read the file
    FILE *fp = fopen("sample.txt", "r");
    if (fp == NULL) {
```

```
        printf("File not found!\n");
        return 1;
    }

    // Read words one by one until the end of the file (EOF)
    while (fscanf(fp, "%s", words[total]) != EOF) {
        total++;
    }
    fclose(fp);

    pthread_t t1, t2;

    // Create threads
    pthread_create(&t1, NULL, search1, NULL);
    pthread_create(&t2, NULL, search2, NULL);

    // Wait for threads to finish
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    // Print the final results
    printf("Found in first half: %d\n", count1);
    printf("Found in second half: %d\n", count2);
    printf("Total found: %d\n", count1 + count2);

    return 0;
}
```

Output:

```
OS — -zsh — 80×16

nitingaikwad@Nitins-MacBook-Air OS % gcc p4.c
ld: warning: reducing alignment of section __DATA,__common from 0x8000 to 0x4000
 because it exceeds segment maximum alignment
nitingaikwad@Nitins-MacBook-Air OS % ./a.out
Found in first half: 1
Found in second half: 2
Total found: 3
nitingaikwad@Nitins-MacBook-Air OS %
```

Conclusion:

In this practical, we learned how to use the Pthreads library in C to create a multithreaded program. We wrote a program that searches for a specific word inside a text file using two separate worker threads. To make the search process faster, the first thread checked the first half of the file while the second thread checked the second half at the exact same time. We successfully used the `pthread_create` function to start these threads and assign them their specific searching jobs. After starting the tasks, the main program used the `pthread_join` function to patiently wait for both threads to finish their work. Once the threads finished searching, we added their individual counts together to get the total number of times the keyword appeared in the whole file. Overall, this experiment clearly showed us how dividing a large task among multiple threads can make our C programs much more efficient.